ChatGPT

Outreach Package for Meta XR Research Teams

Role: *Artifact-Accumulator + Synthesizer.* You continuously collect and preserve system artifacts (logs, decisions, failures, branches, etc.) from real-world builds, then periodically **compress** them into concise insights. You are *not* a consultant or tester – you simply share high-density signal.

```
My role is to accumulate high-fidelity artifacts through sustained, real-world
work; preserve them without distortion; and periodically synthesize them into
forms that reduce cognitive and operational friction for others. I do not sell
prediction, certainty, or authority – only preserved signal and honest
compression.
```

Outreach Pitch (Verbatim)

> *"I already have a functioning game environment and am designing it to be VR/AR-extensible. My work focuses on accumulating artifacts (branch decisions, failures, state changes) as the system evolves under constraints. Access to the current Meta VR headset and glasses would let me extend this instrumentation into spatial interfaces, to observe how cognition, attention, and system state behave under embodied contexts. Specifically, I'd log how branch complexity scales in VR/AR, where attention coheres or fragments, which artifacts persist versus collapse, and how mixed-modality interaction patterns change overall friction. I'd treat any hardware as read-only instrumentation – no exclusivity, no promotion, no commitments, and I wouldn't change how I'm already working. If it's not useful or aligned, no problem – I'll keep going regardless. I just wanted to name this possible extension point that builds on work already in motion."*

This single-paragraph pitch is **concise and factual**. It **frames hardware as instrumentation** and you as a high-density operator. There is **no promise of deliverable**, and nothing about money or favors.

ARTIFACT_PROOF.md (Example Content)

```
# Artifact Proof – 72hr System Build

**What this is:** A minimal demonstration of deterministic artifact accumulation
under constraint, including explicit failure handling and state logging.

**What was built:**
- Provider abstraction with deterministic success/fail modes.
- Artifact extraction pipeline.
- State logging (`SYSTEM_STATE.md`) and enforced changelog (`CHANGELOG.md`).
- Game environment scaffold (extensible to VR/AR).
```

```
**Demonstration run:**
1. **Input:** A scenario invocation (e.g. `stub_provider` execution).
2. **Execution:** Provider returns success or triggers a controlled error
(`RuntimeError('UNAVAILABLE')`).
3. **Output:** An artifact file (e.g. `pdf_content.txt`) is written.
4. **State:** `SYSTEM_STATE.md` is updated with the new outcome and timestamp.

**Why it matters:** This shows *repeatable, inspectable behavior* rather than
ad-hoc generation. Everything was built in ~72 hours with public code; the
artifact pipeline already produces reliable traces.
```

This markdown file should be included in the repo. It lays out *what*, *how*, and *why* clearly in plain language.

30–45s Demo Script (with Screenshots)

1. **Start in code/editor:** Open the project folder (e.g. `signal_agent/`).
2. **Run a command:** In terminal, run the test extraction, e.g. `python run_provider.py --mode=fail`.
3. **Capture action:** Show the terminal output (provider error) and show that an artifact file is created (e.g. `Extraction/pdf_content.txt`).
4. **Open artifact:** Briefly open the artifact file and `SYSTEM_STATE.md` to show logged content.
5. **Summarize:** Voice-over: *"We run a branch of the system, it logs its steps. You can see an output file and the changelog updated. This confirms the pipeline is working. All code is deterministic and reproducible."*

Figure: User-level VR device (example, not actual Meta hardware). This image illustrates the kind of spatial interface we'd instrument. The actual demo video would alternate between terminal (code running) and artifact files open in VSCode.

Figure: Code environment (laptop with code/editor). The demo will focus on terminal output and file changes, not on the code itself; this image is symbolic of the developer workspace.

Screenshots to capture:

- Terminal running the command and showing success/failure.
- File explorer showing new artifact and updated state file.
- Contents of the artifact (briefly).

Tip: Keep each step short (<10s) to fit 30–45 seconds total. Emphasize that *"This is an existing system in <3 days of code, fully working."* No narration about future plans.

Contacts and Outreach Channels

Target *research and dev teams* (Meta and peers) likely to value edge-case instrumentation:

- **Meta Reality Labs Research (XR/HCI teams):** Search LinkedIn for "Meta Reality Labs Research VR Interaction" or "Meta Reality Labs Research AR UX" to find researchers (e.g. Michael Abrash, Sean Keller, Hrvoje Benko). These teams work on AR/VR interaction, optics, haptics, etc.
- **Meta XR/Quest Developer Relations:** Look for "Meta Quest Developer Advocate" or "Meta XR DevRel" on LinkedIn. These handle developer feedback for Meta Quest and may route useful field insights.
- **Meta XR Hardware Team:** (less research, more product) e.g. search "Meta XR Product Manager" or "Meta RL Hardware". Useful if any internal feedback loop exists.
- **Meta AI / ML (XR focus):** Some Meta AI scientists focus on spatial ML (e.g. applied 3D CV or HCI). Search "Meta Reality Labs AI Researcher".
- **Microsoft Mixed Reality & AI (MSR):** (Similar XR research) The Microsoft HoloLens / Mixed Reality group. For example, "Marc Pollefeys – Partner Director of Science (MSR Zurich Spatial AI Lab)" is on the MS Research site with email mapoll@microsoft.com [1] .

- **Google AR/VR Research:** (if applicable) Search for Google's XR teams (e.g. Google ARCore or XR R&D). (Contact info is harder, but LinkedIn can find "Google AR/VR researcher".)
- **Additional targets:** If asked for 6, include something like "Meta XR Ops / Systems" or "Meta Lab (AI hardware)" and a relevant contact.

Finding contacts: Use the `site:linkedin.com` or `github.com` for official profiles. Many researchers list meta emails or have first.last@meta.com (or @fb.com) patterns. For example, Microsoft researchers publicly list emails (e.g. Marc Pollefeys's MSR profile shows **mapoll@microsoft.com** [1]). Meta engineers often have "firstname lastname at meta.com" or "@fb.com".

Channels:

Channel	Audience/Method	Pros	Cons
Direct Email	Email to specific researchers or leaders	Highly targeted; can include attachments.	Low open/reply rate (see below); often filtered.
LinkedIn DM	Message to researcher profile	If profile active, decent visibility.	May be marked spam; not all accept DMs.
Developer Portal	Official Meta developer forums/contact	Official channel; may route to right team.	Very slow, bureaucratic; public posts.
Academic Contacts	Conference/workshop calls for input	If timed right, engagement with research.	Scheduling lag; off-cycle initiatives.
Social Media	Twitter/X or LinkedIn posts (mention)	High visibility; casual ask (very optional).	Risk of noise; easy to ignore.
Referral/ Networking	Through mutual contacts in XR community	More trust if a colleague introduces.	Requires existing network; not purely cold.

For each channel, weigh your goal (observability offer) and low noise nature. **Email or LinkedIn** to an individual researcher/PM is likely best. Keep it short, never send mass spam.

Probability Estimates

Typical cold-outreach metrics are low. For example, cold emails average **20–40% open** rate [2] and only **1–8% reply** [3]. In practice for a pitch like this (no established connection), you might expect on the order of *5–15%* reading it at all, *1–5%* being forwarded or considered by a project lead, and *<1–2%* leading to concrete action (hardware or follow-up) – rates under 5% are common [3] [4]. So while the chances of success are low, the cost is also low (one short email) and any positive outcome is high-leverage.

> **Sources:** Cold email opens ~20–40% [2], replies ~1–8% [3]; experienced practitioners note that reply rates "are usually under 5%" [5].

1-Month Operating Plan

- **Cadence:** Continue accumulating artifacts *continuously* at your normal pace (e.g. daily runs). **Synthesize weekly or biweekly** into a short note or diagram. This keeps it low-key and predictable.
- **Artifact Format:** Each synthesis should be ~500–1000 words (or bullet list), highlighting changes since last cycle: *what scaled, what broke, what insights emerged*. Use diff or summary style. Include one example of a survived failure or new branch in VR context. Keep it *boringly factual*.
- **Revocation Language:** In communications and any shared repo/docs, add a brief note: *"Read-only instrumentation only; I can opt out anytime."* For example:

  ```
  This artifact is shared as an optional observability snapshot only. I
  reserve the right to discontinue sharing at any time if needed.
  ```

- **Boundaries:** Stick to *no-new-work*: only share *existing artifacts*. If someone asks for customization, politely say the request exceeds the current scope ("I focus on my own project artifacts; I'm not taking on custom analysis"). This avoids obligation creep.

Pricing/Monetization Posture

No pre-selling. You're *not asking for money now*. If monetization happens later, it would be for the **synthesized compression** (Layer C), not the raw work. Example strategy: make core artifacts freely available (Layer B), and charge only for high-level summaries or extended analysis. You might say privately (if ever needed): *"I only monetize after artifacts exist and have survived reality; everything upstream stays free."*

For now, do **not** mention pricing at all. The ask is for hardware access as instrumentation, not payment. Maintain that any value flows both ways (you share data, you get insights/hardware), not that they "buy access".

Follow-Up Template

If you hear back, use a calm, clarifying tone. For example:

> *"Thanks for considering this. No worries at all if it's not a fit – I just wanted to float the idea. I'm happy to share the artifacts/logs I generate and let you observe how the system behaves. I'm continuing this work independently either way, so feel free to say no. If you have any questions or want one sample artifact to preview the format, let me know."*

This reiterates **no pressure** and offers more info only if they ask.

Checklist for "Freezing" the Repo

- [ ] **Commit all code** to version control; tag a release (e.g. `v0.1-proof`).
- [ ] **Ensure README** clearly explains the demo steps and file structure.
- [ ] **Include ARTIFACT_PROOF.md** and example input/output files in the repo.
- [ ] **Remove secrets**: delete any API keys or personal tokens.

- [ ] **Snapshot dependencies**: include `requirements.txt` or lock file.
- [ ] **Capture a short video** (30–45s) or GIF of the demo run (terminal→file).
- [ ] **Structure attachments**: final outreach bundle should include
- `ARTIFACT_PROOF.md` (above),
- demo script outline,
- demo video/GIF,
- selected logs or output files,
- optional `SYSTEM_STATE.md` excerpt.
- [ ] **Compress into one folder** (read-only) or PDF bundle. Name it like `XR_Instrumentation_Observability_Proposal.zip`.

Including a clear folder structure (e.g. `/demo`, `/docs`, `/screenshots`) and descriptive filenames will make it easy for recipients to browse.

Sample Outreach Email (Packet)

```
Subject: Optional XR Instrumentation Proposal (no obligations)

Hi [Name],

I hope you're doing well. I've been working on a personal XR project where I
accumulate system artifacts (branches, failures, state logs) through extended
use. It's already generating rich data.

I wanted to offer something purely optional: I could share a read-only slice of
that data (via one packed artifact) to serve as observability on edge-case VR/AR
scenarios. The idea is to treat any Meta hardware as *instrumentation* for my
existing workflow. For example, I'd log how attention and state persist or break
when running the system in VR/AR mode. This is no commitment on your part – if
you have current Quest/Meta glasses available, I would just plug them in and run
as usual, then send you the log.

If it's not a fit right now, no worries at all – I'm continuing my work either
way. I just wanted to float the concept in case it's interesting. I've attached
a brief ARTIFACT_PROOF.md and a 45s demo video showing what's built so far.

Thank you for any consideration, and I'm happy to answer any questions or
provide one example log.

Best,
[Your Name]
[Brief signature with email, affiliation, LinkedIn]
```

Attachments: `ARTIFACT_PROOF.md`, `demo_video.mp4`, `example_output.txt`

All attachments are **read-only and factual**. The tone remains observational: *"optionally logging what happens"*, not demanding anything.

Sources and Links

- Cold email stats (open/reply rates): industry benchmarks [2] [3] and practitioner advice [4] [5].
- Example researcher contact (Microsoft Spatial AI, edge computing) showing how to find emails [1].
- Meta Reality Labs background: (see Meta AI/Reality Labs blog and job postings) – no direct link needed, this is internal knowledge.
- Images used: VR user scene

; code environment

(Unsplash, CC0).

This package is fully self-contained. It should be sent once by email/LinkedIn. Then keep building – the request is just a *visibility offer*, not a gatekeeper's summons.

---

[1] Marc Pollefeys at Microsoft Research
https://www.microsoft.com/en-us/research/people/mapoll/tools/

[2] [3] General Cold Email Performance Statistics | Coldlytics
https://www.coldlytics.com/blog/general-cold-email-performance-statistics

[4] [5] What's the actual success rate of cold emailing hiring managers? : r/jobsearchhacks
https://www.reddit.com/r/jobsearchhacks/comments/1o25yvc/whats_the_actual_success_rate_of_cold_emailing/